
LINUX

Turning WSL ON Through PowerShell

1. Press “Windows” + “R” keys simultaneously to open the Run prompt.
2. Type in “Powershell” and press “Shift“+ “Ctrl” + “Enter” to provide administrative privileges.
3. Type in the following command and press “Enter“

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux
```

fork() in C

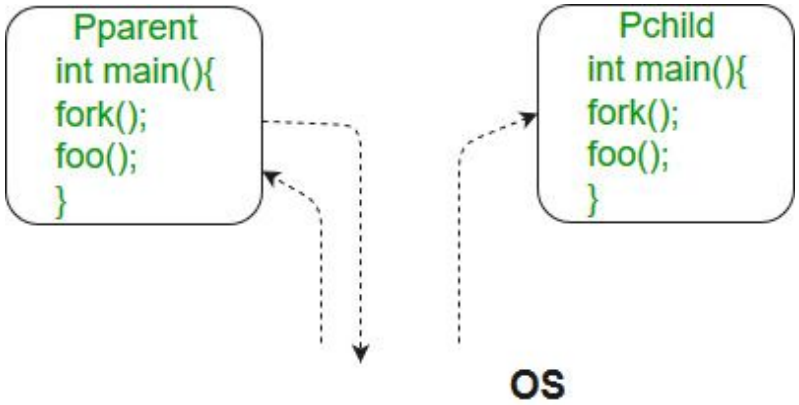
- Fork system call is used for creating a new process, which is called ***child process***, which runs concurrently with the process that makes the fork() call (parent process).

- After a new child process is created, both processes will execute the next instruction following the fork() system call. A child process uses the same pc(program counter), same CPU registers, same open files which use in the parent process.

- It takes no parameters and returns an integer value. Below are different values returned by fork().

fork() in C

- ***Negative Value***: creation of a child process was unsuccessful.
- ***Zero***: Returned to the newly created child process.
- ***Positive value***: Returned to parent or caller. The value contains process ID of newly created child process.



fork() in C

Predict the Output of the following program:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{

        // make two process which run same
        // program after this instruction
        fork();

        printf("Hello world!\n");
        return 0;
}
```

fork() in C

Predict the Output of the following program:

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{

        // make two process which run same
        // program after this instruction
        fork();

        printf("Hello world!\n");
        return 0;
}
```

fork() in C

```
$ gcc -o fork fork.c
```

```
vlukaj@vlukaj:~/OS_Systems$ ls
fork  fork.c
vlukaj@vlukaj:~/OS_Systems$
```

```
$ ./fork
```

```
vlukaj@vlukaj:~/OS_Systems$ ./fork
Hello world!
Hello world!
vlukaj@vlukaj:~/OS_Systems$
```

fork() in C

Calculate number of times hello is printed:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
        fork();
        fork();
        fork();
        printf("hello\n");
        return 0;
}
```

fork() in C

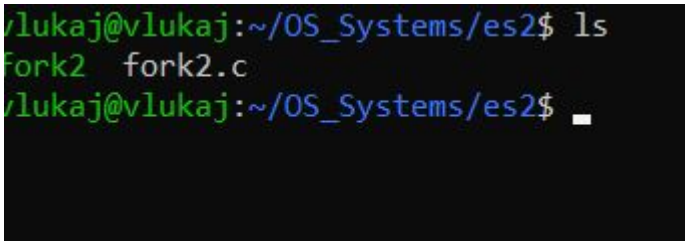
Calculate number of times hello is printed:

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
        fork();
        fork();
        fork();
        printf("hello\n");
        return 0;
}
```

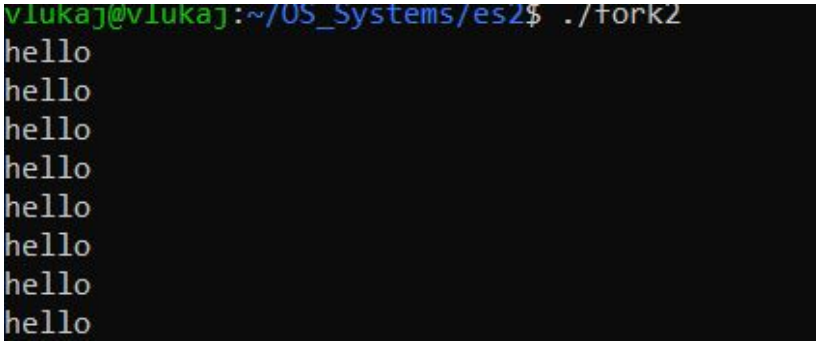
fork() in C

Calculate number of times hello is printed:

```
$ gcc -o fork2 fork2.c
```




```
$ ./fork
```



fok() in C

The number of times ‘hello’ is printed is equal to number of process created. Total Number of Processes = 2^n, where n is number of fork system calls. So here $n = 3$, $2^3 = 8$

Let us put some label names for the three lines:

```
fork ();   // Line 1
fork ();   // Line 2
fork ();   // Line 3

       L1       // There will be 1 child process
    /     \     // created by line 1.
  L2      L2    // There will be 2 child processes
 /  \    /  \   //  created by line 2
L3  L3  L3  L3  // There will be 4 child processes
                // created by line 3
```

fok() in C

So there are total eight processes (new child processes and one original process).

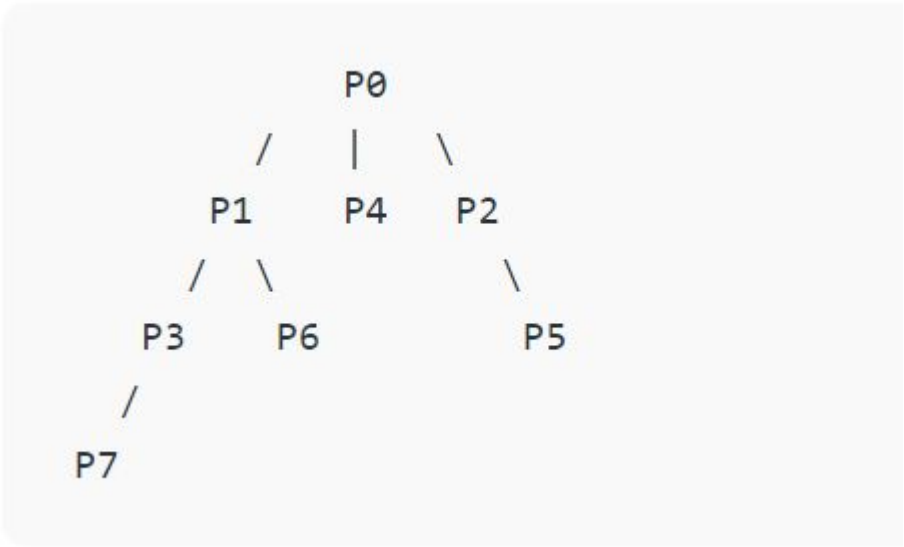
If we want to represent the relationship between the processes as a tree hierarchy it would be the following:

The main process: P0

Processes created by the 1st fork: P1

Processes created by the 2nd fork: P2, P3

Processes created by the 3rd fork: P4, P5, P6, P7



fok() in C

Predict the Output of the following program:

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void forkexample()
{
    // child process because return value zero
    if (fork() == 0)
        printf("Hello from Child!\n");

    // parent process because return value non-zero.
    else
        printf("Hello from Parent!\n");
}
int main()
{
    forkexample();
    return 0;
}
```


```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void forkexample()
{
        // child process because return value zero
        if (fork() == 0)
                printf("Hello from Child!\n");

        // parent process because return value
non-zero.
        else
                printf("Hello from Parent!\n");
}
int main()
{
        forkexample();
        return 0;
}
```

fok() in C

Predict the Output of the following program:

```
$ gcc -o fork fork.c
```

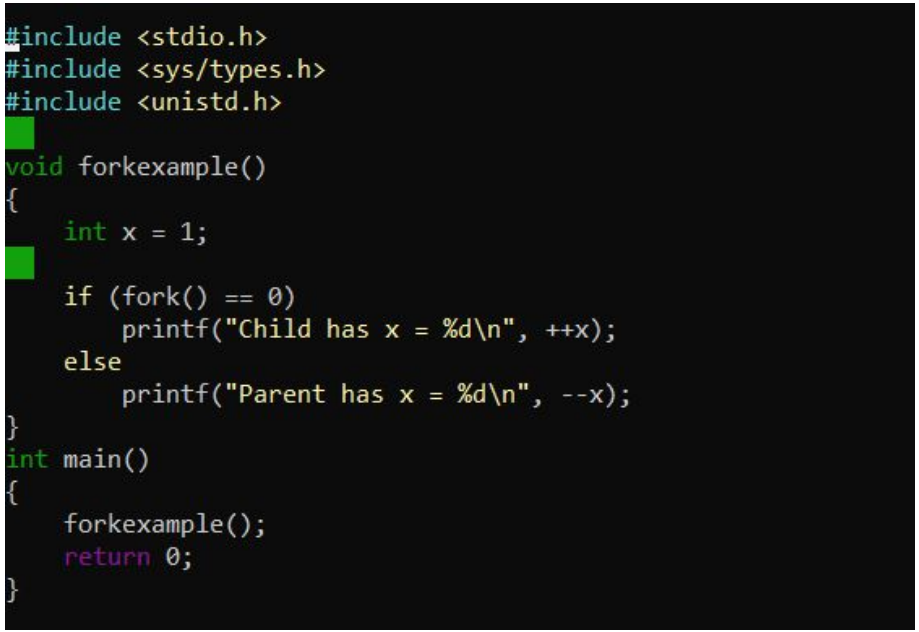
```
vlukaj@vlukaj:~/OS_Systems/es3$ ls
fork  fork.c
vlukaj@vlukaj:~/OS_Systems/es3$
```

```
$ ./fork
```

```
vlukaj@vlukaj:~/OS_Systems/es3$ ./fork
Hello from Parent!
Hello from Child!
```

fok() in C

Predict the Output of the following program:



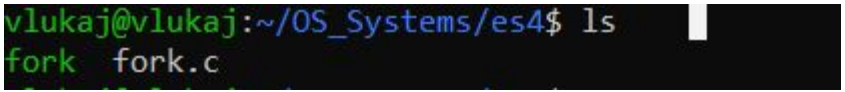
```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

void forkexample()
{
    int x = 1;

    if (fork() == 0)
        printf("Child has x = %d\n", ++x);
    else
        printf("Parent has x = %d\n", --x);
}
int main()
{
    forkexample();
    return 0;
}
```


```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

void forkexample()
{
    int x = 1;

    if (fork() == 0)
        printf("Child has x =
%d\n", ++x);
    else
        printf("Parent has x =
%d\n", --x);
}
int main()
{
    forkexample();
    return 0;
}
```

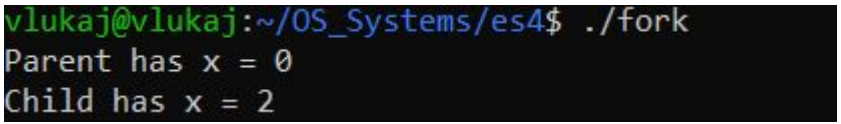
fok() in C

Predict the Output of the following program:

```
$ gcc -o fork fork.c
```




```
$ ./fork
```



Here, global variable change in one process does not affected two other processes because data/state of two processes are different. And also parent and child run simultaneously so two outputs are possible.

fok() vs exec() in C

The fork system call creates a new process. The new process created by fork() is a copy of the current process except for the returned value. The exec() system call replaces the current process with a new program

A process executes the following code:

```
for (i = 0; i < n; i++)
    fork();
```

The total number of child processes created is

(A) n
(B) $2^n - 1$
(C) 2^n
(D) $2^{(n+1)} - 1$

fok() vs exec() in C

```
for (i = 0; i < n; i++)
    fork();
```

The total number of child processes created is

(B) $2^n - 1$

```
          F0        // There will be 1 child process created by first fork
        /    \
      F1      F1    // There will be 2 child processes created by second fork
     /  \    /  \
   F2   F2  F2   F2  // There will be 4 child processes created by third fork
  / \   / \ / \  / \
   ..............    // and so on
```

If we sum all levels of above tree for i = 0 to n-1, we get $2^n - 1$. So there will be $2^n - 1$ child processes. On the other hand, the total number of **process** created are (number of child processes)+1.

fok() vs exec() in C

Consider the following code fragment:

```
if (fork() == 0) {
        a = a + 5;
        printf("%d, %d\n", a, &a);
}
else {
        a = a –5;
        printf("%d, %d\n", a, &a);
}
```

Let u, v be the values printed by the parent process, and x, y be the values printed by the child process. Which one of the following is TRUE?

(A) u = x + 10 and v = y

(B) u = x + 10 and v != y

(C) u + 10 = x and v = y

(D) u + 10 = x and v != y

exec() in C

The exec family of functions replaces the current running process with a new process. It can be used to run a C program by using another C program. It comes under the header file **unistd.h.** There are many members in the exec family which are shown below with examples.

- **execvp** : Using this command, the created child process does not have to run the same program as the parent process does. The **exec** type system calls allow a process to run any program files, which include a binary executable or a shell script .

Syntax:

```
int execvp (const char *file, char *const argv[]);
```

exec() in C

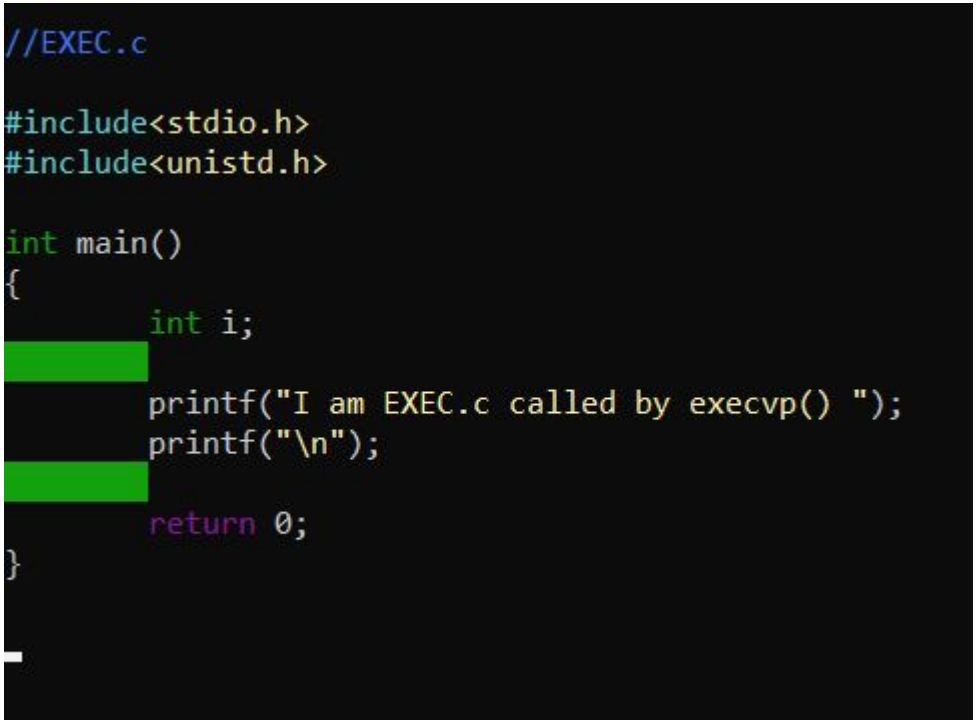
- **file:** points to the file name associated with the file being executed.

  argv: is a null terminated array of character pointers.

  Let us see a small example to show how to use execvp() function in C. We will have two .C files ,

  EXEC.c and **execDemo.c** and we will replace the execDemo.c with EXEC.c by calling execvp() function in execDemo.c .

exec() in C



```c
//EXEC.c

#include<stdio.h>
#include<unistd.h>

int main()
{
        int i;

        printf("I am EXEC.c called by execvp() ");
        printf("\n");

        return 0;
}
```

```c
//EXEC.c

#include<stdio.h>
#include<unistd.h>

int main()
{
        int i;

        printf("I am EXEC.c called by
execvp() ");
        printf("\n");

        return 0;
}
```

exec() in C

```
gcc EXEC.c -o EXEC
```

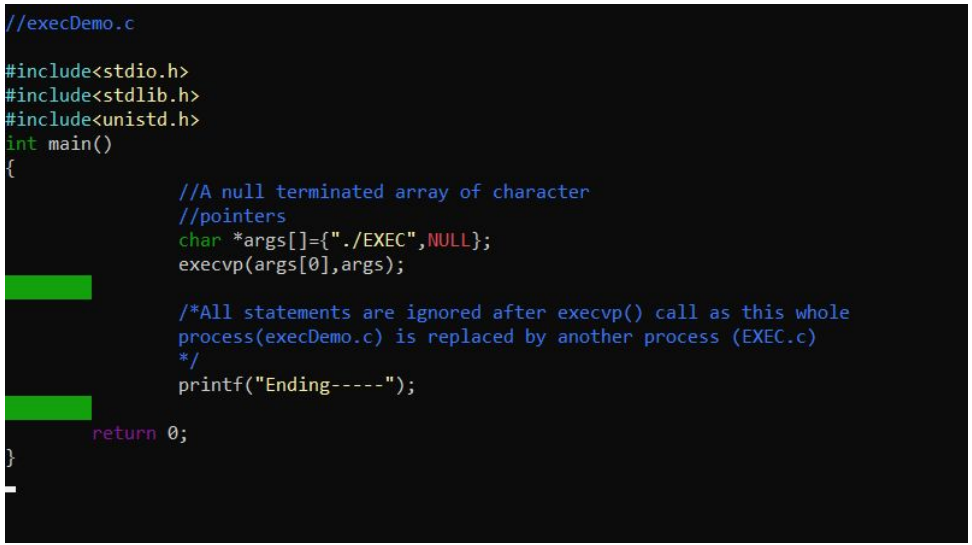
```
vlukaj@vlukaj:~/OS_Systems/es5$ ls
EXEC  EXEC.c
```

```
vlukaj@vlukaj:~/OS_Systems/es5$ ./EXEC
I am EXEC.c called by execvp()
```

```
./EXEC
```

exec() in C

```
//execDemo.c

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int main()
{
		//A null terminated array of character
		//pointers
		char *args[]={"./EXEC",NULL};
		execvp(args[0],args);

		/*All statements are ignored after execvp() call as
this whole
		process(execDemo.c) is replaced by another
process (EXEC.c)
		*/
		printf("Ending-----");

	return 0;
}
```



exec() in C

```
gcc execDemo.c -o execDemo
```

```
vlukaj@vlukaj:~/OS_Systems/es6$ gcc execDemo.c -o execDemo
vlukaj@vlukaj:~/OS_Systems/es6$ ls
execDemo  execDemo.c
```

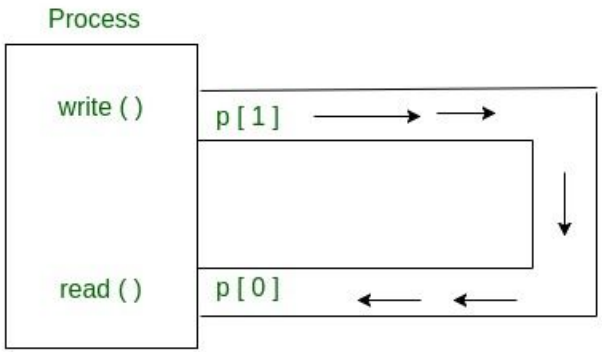
```
vlukaj@vlukaj:~/OS_Systems/es6$ ./execDemo
Ending-----vlukaj@vlukaj:~/OS_Systems/es6$
```

pipe() System call

Conceptually, a pipe is a connection between two processes, such that the standard output from one process becomes the standard input of the other process. In UNIX Operating System, Pipes are useful for communication between related processes(inter-process communication).

- Pipe is one-way communication only i.e we can use a pipe such that One process write to the pipe, and the other process reads from the pipe. It opens a pipe, which is an area of main memory that is treated as a ***“virtual file”***.
- The pipe can be used by the creating process, as well as all its child processes, for reading and writing. One process can write to this “virtual file” or pipe and another related process can read from it.
- If a process tries to read before something is written to the pipe, the process is suspended until something is written.
- The pipe system call finds the first two available positions in the process’s open file table and allocates them for the read and write ends of the pipe.

pipe() System call



Syntax in C language:

```
int pipe(int fds[2]);

Parameters :
fd[0] will be the fd(file descriptor) for the
read end of pipe.
fd[1] will be the fd for the write end of pipe.
Returns : 0 on Success.
-1 on error.
```

Pipes behave **FIFO** (First in First out), Pipe behave like a **queue** data structure. Size of read and write don't have to match here. We can write **512** bytes at a time but we can read only 1 byte at a time in a pipe.

pipe() System call

```
// C program to illustrate
// pipe system call in C
#include <stdio.h>
#include <unistd.h>
#define MSGSIZE 16
char* msg1 = "hello, world #1";
char* msg2 = "hello, world #2";
char* msg3 = "hello, world #3";

int main()
{
        char inbuf[MSGSIZE];
        int p[2], i;

        if (pipe(p) < 0)
                exit(1);
```

```
/* continued */
        /* write pipe */

        write(p[1], msg1, MSGSIZE);
        write(p[1], msg2, MSGSIZE);
        write(p[1], msg3, MSGSIZE);

        for (i = 0; i < 3; i++) {
                /* read pipe */
                read(p[0], inbuf,
MSGSIZE);
                printf("% s\n", inbuf);
        }
        return 0;
}
```

pipe() System call

```c
// C program to illustrate
// pipe system call in C
#include <stdio.h>
#include <unistd.h>
#define MSGSIZE 16
char* msg1 = "hello, world #1";
char* msg2 = "hello, world #2";
char* msg3 = "hello, world #3";

int main()
{
        char inbuf[MSGSIZE];
        int p[2], i;

        if (pipe(p) < 0)
                exit(1);

        /* continued */
        /* write pipe */

        write(p[1], msg1, MSGSIZE);
        write(p[1], msg2, MSGSIZE);
        write(p[1], msg3, MSGSIZE);

        for (i = 0; i < 3; i++) {
                /* read pipe */
                read(p[0], inbuf, MSGSIZE);
                printf("% s\n", inbuf);
        }
        return 0;
}
```

pipe() System call

```
gcc pipe.c -o pipe
```

```
vlukaj@vlukaj:~/OS_Systems/es8$ ./pipe
hello, world #1
hello, world #2
hello, world #3
```

Input-output system calls in C | Create, Open, Close, Read, Write

Important Terminology

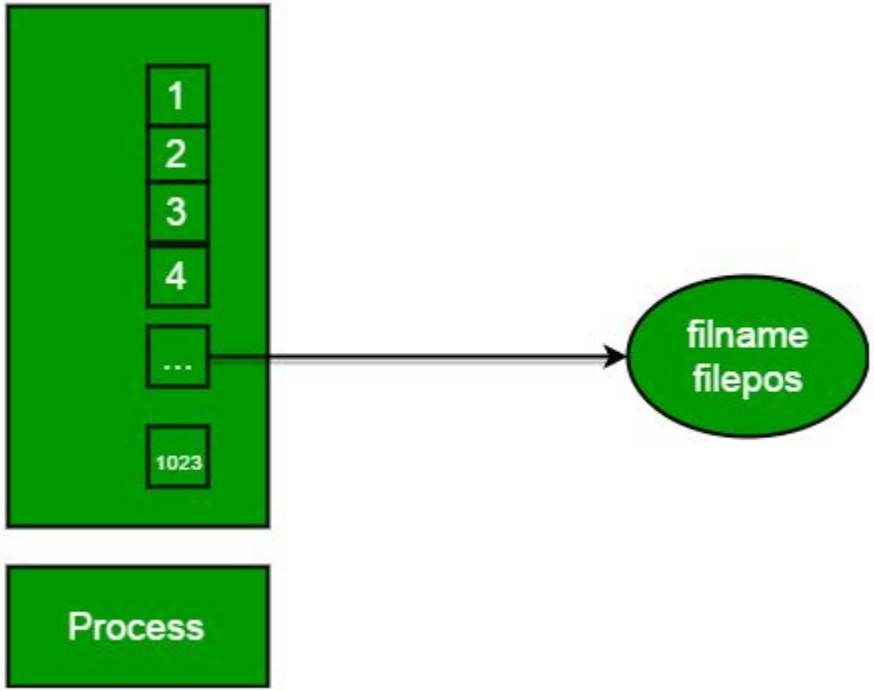
What is the File Descriptor?

File descriptor is integer that uniquely identifies an open file of the process.

File Descriptor table: File descriptor table is the collection of integer array indices that are file descriptors in which elements are pointers to file table entries. One unique file descriptors table is provided in operating system for each process.

File Table Entry: File table entries is a structure In-memory surrogate for an open file, which is created when process request to opens file and these entries maintains file position.

Input-output system calls in C | Create, Open, Close, Read, Write



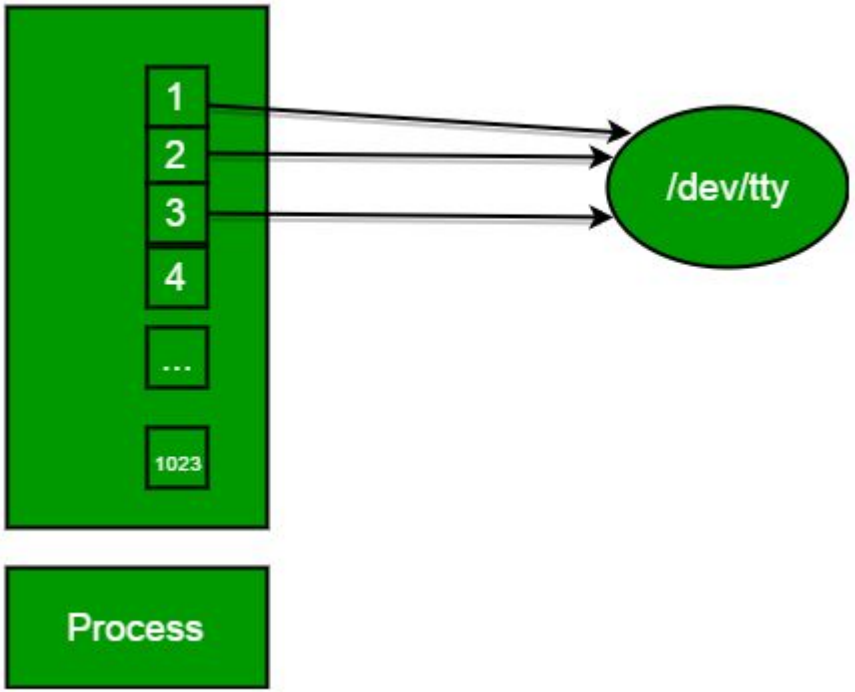
Standard File Descriptors: When any process starts, then that process file descriptors table's fd(file descriptor) 0, 1, 2 open automatically, (By default) each of these 3 fd references file table entry for a file named **/dev/tty**

/dev/tty: In-memory surrogate for the terminal

Terminal: Combination keyboard/video screen

Input-output system calls in C | Create, Open, Close, Read, Write

Read from stdin => read from fd 0 : Whenever we write any character from keyboard, it read from stdin through fd 0 and save to file named /dev/tty.
Write to stdout => write to fd 1 : Whenever we see any output to the video screen, it's from the file named /dev/tty and written to stdout in screen through fd 1.
Write to stderr => write to fd 2 : We see any error to the video screen, it is also from that file write to stderr in screen through fd 2.



Input-output system calls in C | Create, Open, Close, Read, Write

I/O System calls

Basically there are total 5 types of I/O system calls:

1. Create: Used to Create a new empty file.

Syntax in C language:

```
int create(char *filename, mode_t mode)
```

- **filename :** name of the file which you want to create
- **mode :** indicates permissions of new file.

Input-output system calls in C | Create, Open, Close, Read, Write

I/O System calls

Returns:

- return first unused file descriptor (generally 3 when first create use in process because 0, 1, 2 fd are reserved)
- return -1 when error

How it work in OS

- Create new empty file on disk
- Create file table entry
- Set first unused file descriptor to point to file table entry
- Return file descriptor used, -1 upon failure

Input-output system calls in C | Create, Open, Close, Read, Write

I/O System calls

Returns:

- return first unused file descriptor (generally 3 when first create use in process because 0, 1, 2 fd are reserved)
- return -1 when error

How it work in OS

- Create new empty file on disk
- Create file table entry
- Set first unused file descriptor to point to file table entry
- Return file descriptor used, -1 upon failure

Input-output system calls in C | Create, Open, Close, Read, Write

2. open: Used to Open the file for reading, writing or both.

```
Syntax in C language
#include<sys/types.h>
#include<sys/stat.h>
#include <fcntl.h>
int open (const char* Path, int flags [, int mode ]);
```

Input-output system calls in C | Create, Open, Close, Read, Write

Parameters

- **Path:** path to file which you want to use
  - use absolute path begin with "/", when you are not work in same directory of file.
  - Use relative path which is only file name with extension, when you are work in same directory of file.
- **flags :** How you like to use
  - **O_RDONLY**: read only, **O_WRONLY**: write only, **O_RDWR**: read and write, **O_CREAT**: create file if it doesn't exist, **O_EXCL**: prevent creation if it already exists

How it works in OS

- Find the existing file on disk
- Create file table entry
- Set first unused file descriptor to point to file table entry
- Return file descriptor used, -1 upon failure

Input-output system calls in C | Create, Open, Close, Read, Write

```
// C program to illustrate
// open system call
#include<stdio.h>
#include<fcntl.h>
#include<errno.h>
extern int errno;
int main()
{
        // if file does not have in directory
        // then file foo.txt is created.
        int fd = open("foo.txt", O_RDONLY | O_CREAT);

        printf("fd = %d\n", fd);

        if (fd ==-1)
        {
                // print which type of error have in a code
                printf("Error Number % d\n", errno);

                // print program detail "Success or failure"
                perror("Program");
        }
        return 0;
}
```

Input-output system calls in C | Create, Open, Close, Read, Write

3. close: Tells the operating system you are done with a file descriptor and Close the file which pointed by fd.

```
Syntax in C language
#include <fcntl.h>
int close(int fd);
```

Parameter:

- **fd** :file descriptor

Return:

- **0** on success.
- **-1** on error.

Input-output system calls in C | Create, Open, Close, Read, Write

```
// C program to illustrate close system Call
#include<stdio.h>
#include <fcntl.h>
int main()
{
        int fd1 = open("foo.txt", O_RDONLY);
        if (fd1 < 0)
        {
                perror("c1");
                exit(1);
        }
        printf("opened the fd = % d\n", fd1);

        // Using close system Call
        if (close(fd1) < 0)
        {
                perror("c1");
                exit(1);
        }
        printf("closed the fd.\n");
}
```

Input-output system calls in C | Create, Open, Close, Read, Write

4. read: From the file indicated by the file descriptor fd, the read() function reads cnt bytes of input into the memory area indicated by buf. A successful read() updates the access time for the file.

```
Syntax in C language
size_t read (int fd, void* buf, size_t cnt);
```

Parameters:

- **fd:** file descriptor
- **buf:** buffer to read data from
- **cnt:** length of buffer

Input-output system calls in C | Create, Open, Close, Read, Write

Returns: How many bytes were actually read

- return Number of bytes read on success
- return 0 on reaching end of file
- return -1 on error
- return -1 on signal interrupt

Important points

- **buf** needs to point to a valid memory location with length not smaller than the specified size because of overflow.
- **fd** should be a valid file descriptor returned from open() to perform read operation because if fd is NULL then read should generate error.
- **cnt** is the requested number of bytes read, while the return value is the actual number of bytes read. Also, some times read system call should read less bytes than cnt.

Input-output system calls in C | Create, Open, Close, Read, Write

```
// C program to illustrate
// read system Call
#include<stdio.h>
#include <fcntl.h>
int main()
{
int fd, sz;
char *c = (char *) calloc(100, sizeof(char));

fd = open("foo.txt", O_RDONLY);
if (fd < 0) { perror("r1"); exit(1); }

sz = read(fd, c, 10);
printf("called read(% d, c, 10). returned that"
                " %d bytes were read.\n", fd, sz);
c[sz] = '\0';
printf("Those bytes are as follows: % s\n", c);
}
```

Input-output system calls in C | Create, Open, Close, Read, Write

```
// C program to illustrate
// read system Call
#include<stdio.h>
#include <fcntl.h>
int main()
{
int fd, sz;
char *c = (char *) calloc(100, sizeof(char));

fd = open("foo.txt", O_RDONLY);
if (fd < 0) { perror("r1"); exit(1); }

sz = read(fd, c, 10);
printf("called read(% d, c, 10). returned that"
                " %d bytes were read.\n", fd, sz);
c[sz] = '\0';
printf("Those bytes are as follows: % s\n", c);
}
```